DISTRIBUTED AUTONOMOUS SURVEILLANCE AND RESCUE ARCHITECTURE INTEGRATING COMPUTER VISION WITHIN MULTI-AGENT DRONE COORDINATION

BS ELECTRICAL ENGINEERING

Session: 2024 - 2025

By

Muhammad Huraira Anwer	***Roll No: 25100314***
Muhammad Moeez	***Roll No: 25100192***
Sadaan Tahir	***Roll No: 25100224***
Raafay Saeed Kazmi	***Roll No: 25100197***

Advisor: Dr Hassan Jaleel,
Assistant Professor, EE.

Co-Advisor(s): Dr Murtaza Taj,
Associate Professor, CS.

DEPARTMENT OF ELECTRICAL ENGINEERING,
SYED BABAR ALI SCHOOL OF SCIENCE AND ENGINEERING,

LAHORE UNIVERSITY OF MANAGEMENT SCIENCES,
PAKISTAN

PROJECT ABSTRACT

Our project focuses on the development and deployment of a distributed multi-agent drone system for surveillance and rescue by utilizing computer vision and autonomous path planning. The rational behind our project stems from the growing demand for autonomous systems for search-and-rescue operations. Such Autonomous drone systems provide a scalable solution, capable of covering large areas more efficiently.

The motivation of our project is to augment the efficiency of search-and-rescue missions by utilizing multi-agent systems to improve the area coverage, response times, and allow for autonomous decision-making. Additionally, the project aims to illustrate how drones, when coordinated through sophisticated algorithms, can offer a scalable solution for real-world missions, such as locating victims in disasters or performing large-scale surveillance tasks.

The system is modelled for simulation using Robot Operating System (ROS) and Gazebo. These tools provide a stable environment for testing and validating algorithms in real time before deploying hardware. The simulation includes Ardupilot for drone control and MAVProxy for ground control. Python scripts are used to interact with the drones allowing the testing of path planning and multi-agent cooperation.

The proposed system uses a hierarchical framework. A master drone performs a swift initial aerial survey of the area, utilizing computer vision to detect a defined flag. During this reconnaissance, aerial images and object detection parameters are collected and processed into a heatmap, illustrating areas of high target probability. This heatmap is subsequently relayed to a swarm of smaller, low-altitude drones, which are dispatched to investigate these high-probability regions.

For path planning, the smaller drones use a gradient ascent algorithm to autonomously navigate towards regions with the highest probability of target presence. To avoid ensure effective coverage of each drone to a flag, a potential field algorithm is integrated. This not only mitigates risk of drone collisions but also directs the drones towards distinct

targets, facilitating a balanced and equilibrium-based deployment. Each drone then conducts precise localization of the targets, enhancing the search to pinpoint exact locations.

ACKNOWLEDGMENTS

The authors wish to express sincere appreciation to **Ibrahim Rana** and **Muhammad Ahson Hassan** for their invaluable support and guidance regarding the hardware setup and overall understanding of the project. Their contributions were instrumental during the development and implementation phases of this senior project.

ORIGINALITY CERTIFICATE

"We the undersigned certify that this submission is the original work of members of the group and meets the Faculty's Expectations of Originality":

1. Sadaan Tahir [2025-10-0224]

   Signature: _____ 25100224@lums.edu.pk

2. Raafay Saeed Kazmi [2025-10-0197]

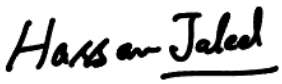
   Signature: _____ 25100197@lums.edu.pk

3. Muhammad Moeez [2025-10-0192]

   Signature: _____ 25100192@lums.edu.pk

4. Muhammad Huraira Anwer [2025-10-0314]

   Signature: _____ 25100314@lums.edu.pk

Dr. Hassan Jaleel 

_____

hassan.jaleel@lums.edu.pk

Dr. Murtaza Taj

_____

murtaza.taj@lums.edu.pk

Table of Contents

PROJECT ABSTRACT ..... **II**

ACKNOWLEDGMENTS ..... **IV**

ORIGINALITY CERTIFICATE ..... **V**

LIST OF FIGURES ..... **VIII**

LIST OF TABLES ..... **X**

Chapter 1 ..... **1**

Introduction 1
1.1: Motivation 1
1.2: Problem Statement 2
1.3: General Block Diagram 2
1.4: Social Benefits And Relevance 4
1.5: Goals 4
1.6: Objectives 5
1.7: Outcomes 5
1.8: Deliverables (Outputs) 5
1.9: Timeline And Distribution of Work 6

Chapter 2 ..... **9**

Background 9
2.1: Literature Review 6
2.2: Market Survey (If applicable) 6

Chapter 3 ..... **14**

Methodology and Tools 14
3.1: System Level Design 14
3.2: Final Design 17
3.3: Tools/Instruments 17

Chapter 4 ..... **19**

Project Simulation 19
4.1: Simulation Setup 19
4.2: Simulation Results 24
4.3: Result Analysis And Outlook 9

Chapter 5 ..... **26**

Project Implementation 26
5.1: Hardware Setup 26
5.2: Experimental Procedure 26

5.3: Implementation Results 35
5.4: Result Analysis And Outlook 38

Chapter 6 ..... **42**

Cost Analysis 42

Chapter 7 ..... **46**

Conclusion and Future Recommendations 46
References 46

LIST OF FIGURES

FIG 1 BLOCK DIAGRAM 3

FIG 2 GANNT CHART FOR FALL 2024 6

FIG 3 GANNT CHART FOR SPRING 2025 6

FIG 4 ORTHOGRAPHIC VIEW OF SIMULATION ENVIRONMENT 21

FIG 5 ANNOTATED SIMULATION ENVIRONMENT 21

FIG 6 A RED CUBE USED AS A MARKER TO TEST BASIC COLOR DETECTION 22

FIG 7 TEXTURED MARKER FOR THE SIMULATION 22

FIG 8 MARKER DATASET CREATED USING ROBOFLOW 22

FIG 9 TESTING CUSTOM TRAINED YOLOV8 MODEL. 87% ACCURACY IS ACHIEVED WITH ONLY 40 TRAIN IMAGES ON THE TEST IMAGE SHOWN. 23

FIG 10 SYSTEMATIC FLOW DIAGRAM OF OUR SIMULATION ARCHITECTURE 23

FIG 11 LOCATION OF MARKERS ON GRIDDED ARENA AND GENERATED HEATMAP BY LEADER DRONE 24

FIG 12 DRONE 2 DETECTS ITS TARGETED MARKER 25

FIG 13 DRONE 1 DETECTS ITS TARGETED MARKER 25

FIG 14 DRONE 3 DETECTS ITS TARGETED MARKER 25

FIG 15 SYSTEMATIC FLOW DIAGRAM FOR THE HARDWARE IMPLEMENTATION 27

FIG 16 LEADER DRONE (500MM FRAME TYPE) 28

FIG 17 FOLLOWER DRONES (350MM FRAME TYPE) 29

FIG 18 ABS SHEET COVER 30

FIG 19 THE CONSTRUCTED MARKER WITH OUR CUSTOM TEXTURE 30

FIG 20 THE RED SQUARE MARKS THE BOUNDARIES OF OUR TESTING AREA 31

FIG 21 EXPERIMENTAL SETUP 31

FIG 22 DIAGRAM ILLUSTRATING BOUSTROPHEDON PATH..... 33

FIG 23 DIAGRAM ILLUSTRATING COMMUNICATION BETWEEN DRONES 34

FIG 24 MODEL PERFORMANCE 36

FIG 25 MARKER DETECTION..... 36

FIG 26 HEATMAP GENERATED BY LEADER DRONE 37

LIST OF TABLES

TABLE 1 DISTRIBUTION OF WORK 7

TABLE 2 ELECTRONIC COMPONENTS FOR EACH DRONE TYPE 27

TABLE 3 LEADER DRONE PERFORMANCE 39

TABLE 4 FOLLOWER DRONES PERFORMANCE 40

TABLE 5 COLOR LANDING PERFORMANCE 40

TABLE 6 IMPORTED DRONE PARTS LIST 42

TABLE 7 LOCAL DRONE PARTS LIST 43

TABLE 8 OTHER ADDITIONAL COMPONENTS 44

Chapter 1

INTRODUCTION

1.1: MOTIVATION

The increasing frequency of unexpected emergencies stresses the crucial need for rapid and effective response mechanisms to identify any potential people who could be harmed or need to be rescued. Traditional search techniques using human resources may take time and depend on the ease of the terrain or the timing it furnishes. For instance, in a disaster circumstance like an earthquake or a flood, time is of the essence, and finding out what happened precisely is critical. Moreover, this urgency propels our pursuit of developing a distributed drone system for search operations.

The project will use sensor-rich drones that can use autonomous navigation to completely change and modernize the process of searching large areas. Instead of helicopters or manual searches that are very resource-intensive and do not cover much ground, drones can cover large distances very quickly and will pinpoint specific targets. Concentrating on flat terrain and strategically placed flags gives us an accurate simulation of real-world search conditions; this helps prove the system's effectiveness in such use cases.

This project is not just a reading task but a stride in meeting the urgent needs of society. Increased search efficiency leads to improved public safety and preparation for emergencies. Integrating heatmap technology into dynamic navigation guarantees our drones' ability to adjust and accommodate changes on the ground, serving as an essential tool for first responders. Through the implementation of projects of this nature, we hope that a more resilient system will be developed in dealing with crises to make our communities safe and prepared for anything unexpected or for simple searching operations.

Although the present attention of our system is to simulate search and rescue operations, its later potential is not limited to this scope alone. For example, in agriculture applications, drones can survey huge fields by analyzing data from their sensors to note early signs of fire or crop disease,

thus managing those issues proactively and preserving resources. Moreover, for local communities, like our university campus, this system would also be useful for locating missing people or determining places that need assistance in terms of safety and security. Through highlighting these practical use cases, we highlight the power of our system to address real-world problems in different spheres.

1.2: PROBLEM STATEMENT

Our project aims to develop a distributed multi-agent drone system capable of conducting efficient surveillance and rescue operations. This will be achieved by focusing on the following key areas:

- Autonomous navigation and coordination among multiple drones.
- Real-time data collection and processing using computer vision.

1.3: GENERAL BLOCK DIAGRAM

Please provide a higher level block diagram which could be understood easily by professionals not necessarily related to your own field.Click or tap here to enter text.

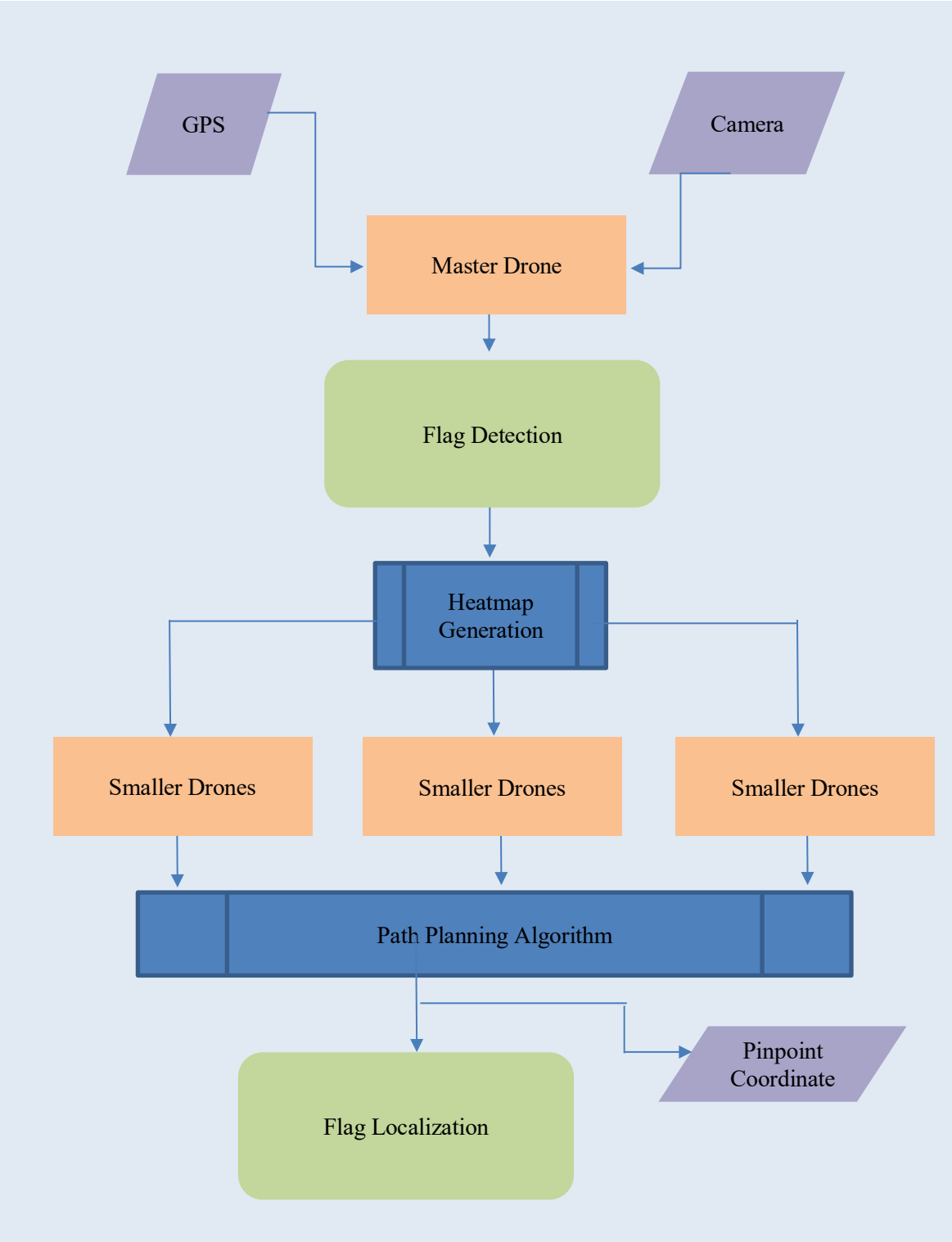


Fig 1 Block Diagram

1.4: SOCIAL BENEFITS AND RELEVANCE

A distributed autonomous surveillance and rescue architecture based on multi-agent drone systems will offers substantial societal benefits by increasing emergency response capabilities and public safety. It will be able to localize targets with real time data processing, significantly improving response time and operational effectiveness compared to traditional methods of deploying human resources and helicopters.

This also improves robotics and computer vision educational opportunities under Sustainable Development Goal 4 (Quality Education). By integrating these technologies in the learning environments, students can be encouraged to go into research and development of technological innovation.

Although drone technology raises ethical issues, especially privacy, we can make stricter rules to protect individual's rights and responsible use. Proper rules and regulations can address data security and airspace issues.

Furthermore, the project supports Sustainable Development Goal 9 (Industry, Innovation and Infrastructure) by promoting innovation in emergency response systems. The multi-agent drones application exemplifies how technology can be used to solve specific challenges and improve infrastructure by providing improved approach to traditional methods.

In general, all these points show the broader societal impact of our project, how it can contribute to sustainable development.

1.5: GOALS

Our project is centered on the development of a distributed decentralized drone system designed to operate efficiently across diverse environments. This innovative system will enable drones to communicate and coordinate their movements, making it particularly well-suited for large-area search and rescue operations. Additionally, we aim to enhance these drones with advanced

sensors, including cameras and GPS, to facilitate autonomous navigation. Rather than following predetermined routes, the drones will utilize heatmaps to dynamically identify and navigate to areas of interest. Ultimately, we aspire to create a reliable drone system that assists in monitoring expansive areas during emergencies, significantly improving response times while prioritizing safety and operational efficiency.

1.6: OBJECTIVES

The project initially aims to prepare and integrate the hardware setup and devices (Jetson Nano, Odroid Xu4s, Flight Controllers) with the relevant packages and modules so they are able to run basic Python scripts to facilitate autonomous drone flights while receiving correct GPS and IMU data. Following the hardware setup phase, we will undertake two months of applying the simulations based on the Robot Operating System (ROS) and Gazebo to the actual hardware. This stage will be particularly directed at forming and realizing computer vision algorithms enabling the master drone to perform real time scanning in various conditions. Such algorithms will allow smaller drones to find and reach the target autonomously.

Simultaneously with the second phase, we will begin setting up the communication protocols and methods for the drones. This part involves setting up a router, and socket-based Python communications between the drones so they can share their positions resulting in an efficient convergence of the algorithm with minimal overlap in their paths.

1.7: OUTCOMES

This distributed drone technology when implemented will improve the response to emergencies and community resilience. The system could improve search and rescue processes since it streamlines coordination and communication between rescue drones, assuring total coverage of targeted areas. Development will also increase safety for both the operators and casualties of the emergency, as operations in dangerous conditions will be conducted by drones instead of people. Moreover, the use of advanced sensors and autonomous navigation will facilitate accurate and timely interventions during emergencies. Finally, the results of the investigation will contribute

to the existing post disaster management systems on how to minimize the effect of disasters in communities and promote quicker recovery.**1.8: Deliverables (Outputs)**

NONE FOR NOW

1.9: TIMELINE AND DISTRIBUTION OF WORK

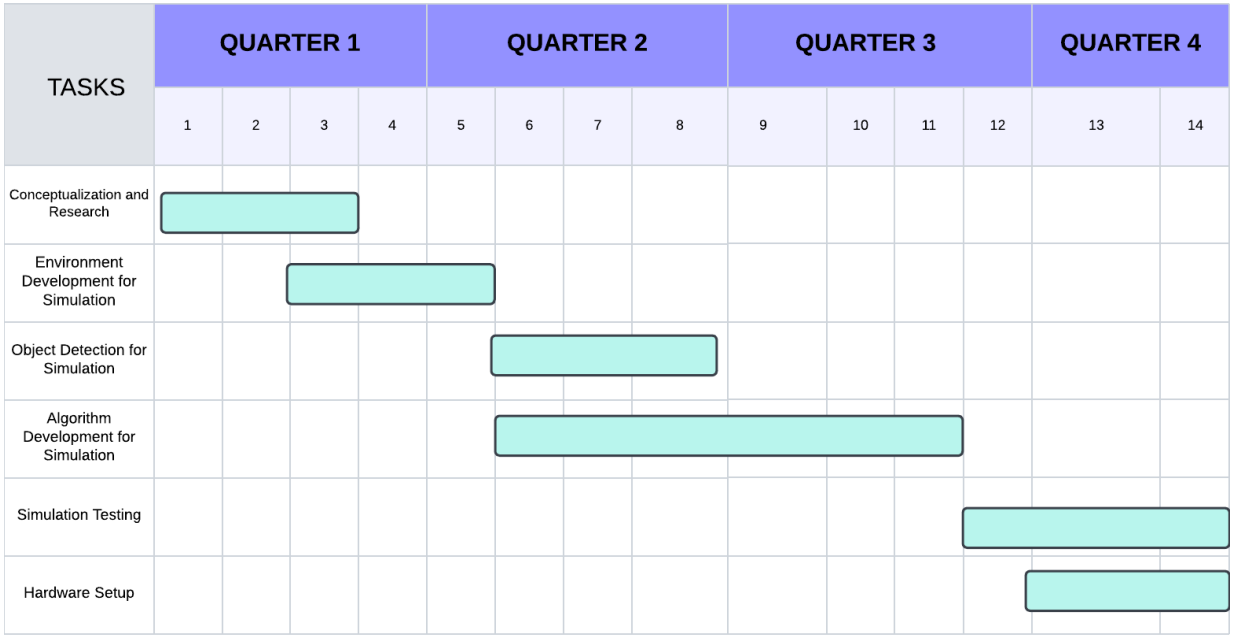


Fig 2 Gannt Chart for Fall 2024

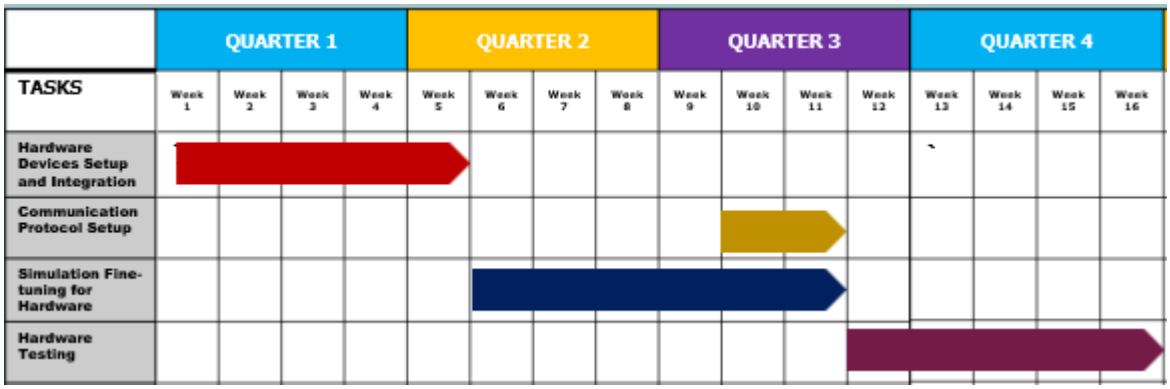


Fig 3 Gannt Chart for Spring 2025

Table 1 DISTRIBUTION OF WORK

Task	Responsible Person	Planned Duration
Conceptualization and Research	Muhammad Huraira Anwer Sadaan Tahir Muhammad Moeez Raafay Saeed Kazmi	Three Weeks
Environment Development for Simulation	Muhammad Huraira Anwer Sadaan Tahir	Three weeks
Object Detection for Simulation	Sadaan Tahir	Three Weeks
Algorithm Development for Simulation	Muhammad Huraira Anwer Muhammad Moeez Raafay Saeed Kazmi	Six weeks
Simulation Testing	Muhammad Huraira Anwer Muhammad Moeez Sadaan Tahir Raafay Saeed Kazmi	Three Week
Hardware Setup	Muhammad Moeez Sadaan Tahir Raafay Saeed Kazmi	Two Week
Hardware Devices Setup and Integration	Muhammad Huraira Anwer Sadaan Tahir	Five weeks

Communication Protocol Setup	Muhammad Moeez Rafay Saeed Kazmi	Two Weeks
Simulation Algorithm Fine-tuning for Hardware	Muhammad Huraira Anwer Muhammad Moeez Raafay Saeed Kazmi	Six weeks
Hardware Testing	Muhammad Huraira Anwer Muhammad Moeez Sadaan Tahir Raafay Saeed Kazmi	Three Weeks

Chapter 2

BACKGROUND

2.1: LITERATURE REVIEW

2.1.1 Introduction to Search and Rescue Challenges

Natural disasters, ranging from devastating earthquakes and widespread floods to destructive wildfires and unpredictable tsunamis, continue to inflict immense human suffering and economic losses globally. In the immediate aftermath of such catastrophic events, the critical window for saving lives is often narrow, making the timely and efficient localization of missing people an absolute imperative to minimize fatalities. While many formidable challenges frequently hamper heroic and indispensable traditional search and rescue (SAR) operations. These include the sheer vastness of affected areas, rendering manual ground searches painstakingly slow and resource-intensive; the presence of difficult-to-access terrains, such as collapsed structures, dense forests, or treacherous waterways, which pose significant physical barriers to human rescuers; and the frequent disruption or complete collapse of conventional communication networks, severely impeding coordination and information flow among rescue teams. Furthermore, SAR personnel are inherently exposed to substantial risks, including unstable environments, hazardous materials, and the emotional toll of witnessing widespread devastation. These inherent complexities and limitations of conventional methods underscore the urgent and growing need for advanced, scalable, and autonomous technological solutions to augment and enhance conventional SAR efforts, ensuring a more rapid, safer, and comprehensive response.

2.1.2 Evolution of Drone-Based Search and Rescue

The rapid advancements in Unmanned Aircraft Systems (UAS), colloquially known as drones, have ushered in a new era for various critical applications, most notably in emergency response and disaster management. Drones offer unparalleled advantages in SAR operations that directly address traditional methods' limitations. Their ability to rapidly cover extensive areas, often inaccessible or dangerous for human teams, significantly boosts search efficiency. Equipped

with diverse payloads, drones can navigate hazardous environments, including areas with unstable ground, active fires, or floodwaters, without endangering human rescuers **[4]**. Moreover, they provide real-time aerial reconnaissance, offering an invaluable bird's-eye view of the disaster zone, which is crucial for situational awareness and strategic planning. Early real-world applications have already demonstrated their practical utility and life-saving potential. For instance, drones have been successfully deployed to confirm fatalities in dangerous slot canyons, thereby eliminating the need for risky manual rappels by rescue personnel at night **[8]**. Similarly, they have been used to survey and clear difficult-to-access terrains, allowing ground teams to focus their efforts on more critical or easily reachable areas **[8]**. This growing adoption, driven by technological improvements such as longer flight times, enhanced sensor capabilities (e.g., thermal cameras for night operations or detecting heat signatures), and increased autonomy, highlights the increasing demand for and reliance on autonomous systems in critical humanitarian missions. Integrating drones into SAR protocols represents a transformative step towards more effective and safer disaster response.

2.1.3 Multi-Agent Systems and Coordination in Drone Operations

While the benefits of single drones in SAR are considerable, the true transformative potential for enhancing efficiency, coverage, and resilience lies in the coordinated deployment of multi-agent drone systems or swarms. A collective of drones operating in unison can achieve objectives far beyond the capabilities of a solitary unit. This includes the ability to collectively cover significantly larger areas more quickly, process and collect data in parallel, and enhance overall mission resilience through redundancy, where the failure of one unit does not cripple the entire operation. However, effectively coordinating multiple autonomous vehicles in dynamic, uncertain, and often unstructured disaster environments presents significant technical challenges. These challenges primarily revolve around sophisticated path planning to ensure optimal coverage and minimize search time, robust collision avoidance mechanisms to prevent drones from impacting each other or environmental obstacles, and efficient distributed decision-making to adapt to evolving situations without centralized control **[2]**. Extensive research into multi-vehicle path planning has explored various optimization methods designed to derive team-optimal solutions. These methods must account for real-time environmental uncertainties, such as emerging obstacles or changing weather patterns, while ensuring high

computational efficiency to support real-time operations **[2]**. Therefore, algorithms for cooperative control of multi-agent systems are crucial, as they dictate how individual agents contribute effectively to a shared global objective, such as comprehensive area mapping or target localization. For instance, distributed trajectory planning algorithms, coupled with robust communication infrastructures (e.g., WiFi-based systems), have been successfully implemented and flight-tested even in adversarial environments, demonstrating the practical feasibility of real-time coordination among multiple UAVs **[3]**. Furthermore, theoretical frameworks, such as those for converging multi-agent projected stochastic gradient algorithms, provide a rigorous mathematical basis for understanding how distributed agents can collectively converge towards optimal solutions in complex non-convex optimization problems. This theoretical understanding directly applies to designing efficient distributed path planning and resource allocation strategies within a drone swarm, ensuring that the collective behavior is optimized for the SAR mission **[5]**. Complementary concepts like log-linear learning also offer valuable insights into how distributed agents can learn and adapt their actions to achieve collective goals, even with limited information exchange and asynchronous updates. This ensures the system moves towards a desirable collective state through decentralized intelligence **[6]**. These advanced coordination mechanisms are not merely theoretical constructs but vital tools for ensuring that a swarm of SAR drones can efficiently sweep a designated area, dynamically avoid static and moving obstacles, and effectively share critical information to pinpoint targets with precision.

2.1.4 Role of Computer Vision in Autonomous Surveillance

A critical and indispensable component of autonomous SAR drones is their ability to intelligently interpret environmental data, primarily through sophisticated computer vision systems. This involves equipping drones with high-resolution cameras (both optical and thermal) and advanced algorithms capable of detecting, identifying, and classifying objects of interest, such as human victims, specific distress markers, or signs of life in challenging environments. The effectiveness of computer vision in SAR relies heavily on robust object detection and recognition capabilities, especially given the often cluttered and chaotic nature of disaster scenes.

Significant research efforts have been dedicated to developing highly efficient and accurate algorithms tailored explicitly for drone surveillance in disaster scenarios, focusing on human and action detection. A key enabler for the success of these algorithms is the availability of substantial and diverse datasets, which are crucial for training deep learning models, such as Convolutional Neural Networks (CNNs), to recognize subtle patterns and features indicative of human presence or activity **[7]**. Models inspired by state-of-the-art architectures like SSD (Single Shot MultiBox Detector) have demonstrated promising results in detecting human instances from aerial imagery, achieving high mean Average Precision (mAP) values even in complex backgrounds **[7]**. Beyond purely visual detection, innovative approaches exemplified by SARDO demonstrate the profound potential for victim localization by leveraging mobile phone signals combined with advanced machine learning techniques. This offers a powerful alternative or complementary method to purely visual detection, particularly valuable when victims may be obscured from direct sight due to debris, foliage, or darkness **[1].** The seamless integration of such advanced computer vision capabilities allows SAR drones to autonomously identify and prioritize areas of interest, filter out irrelevant information, and ultimately significantly accelerate the search process, transforming raw visual data into actionable intelligence for rescue teams.

2.1.5 Problem Statement and Research Gap

While the foundational utility of drones in search and rescue operations is now well-established, and the theoretical and practical advancements in multi-agent coordination techniques continue to progress rapidly, a significant and critical research gap persists. This gap lies in the seamless, robust, and scalable integration of sophisticated real-time computer vision capabilities with fully autonomous multi-agent drone coordination for dynamic search and rescue operations. Existing solutions often focus on optimizing single-drone functionalities (e.g., advanced object detection on one drone) or developing multi-agent systems with less emphasis on intelligent, real-time object detection and adaptive path planning directly driven by visual cues. Our project aims to address the core challenge of developing a comprehensive architecture where a lead drone not only intelligently processes visual data to create a dynamic, high-fidelity search map (heatmap) but also autonomously and adaptively guides a coordinated swarm of smaller, agile drones. These follower drones must then precisely cover high-

probability areas identified by the lead drone, while simultaneously ensuring dynamic collision avoidance among themselves and with environmental obstacles, and optimizing for efficient coverage in a truly distributed manner. Our project aims to bridge this critical gap by developing a holistic distributed multi-agent drone system that leverages advanced computer vision for precise target identification and dynamically utilizes sophisticated coordination algorithms for autonomous path planning and optimal area coverage in complex, evolving disaster environments. This integrated approach promises to enhance the speed, accuracy, and safety of SAR missions beyond current capabilities.

Additionally, in order to successfully execute our simulations in the ROS and Gazebo interface, we have taken inspiration from a multitude of GitHub repositories **[9][10][11]**. These open-source code files have been crucial for us to create the necessary environment in Gazebo and use MavROS to pilot our drones effectively. We learned the method to integrate ROS onto a single UAV **[9]**, how to simulate a single UAV in the Gazebo environnement **[10]**, and how to make a custom world in Gazebo **[11]**. These have served as the foundation for all the future work we have done in the simulation, enabling us to achieve our objective effectively.

2.2: MARKET SURVEY

A broader market survey was not directly applicable to our work because the focus was on developing and validating core functionalities.

Chapter 3

METHODOLOGY AND TOOLS

3.1: SYSTEM LEVEL DESIGN

The system-level architecture consisted of two combined elements: a theoretical game-theoretic model (evaluated via simulation) and a distributed hardware implementation. After rigorous testing, the final hardware deployment employed a deterministic Distributed Gradient Ascent (DGA) algorithm to replace the stochastic Bilinear Log-Linear (BLL) method, focusing on computational efficiency and practicality for multi-UAV coordination.

3.1.1 Theoretic Approach

3.1.1.1 Problem Overview

We developed a game-theoretical multi-agent model for several UAVs to effectively capture flags in a specified arena. The square or rectangular arena was sectioned into a grid of square sectors. We employed a square arena in our simulations, but for experiments on the physical platform, we used the SSE outdoor lawn and cricket ground in LUMS. Several flags were placed on the arena, and our UAVs operated to find and capture them. A leader drone provided probabilistic estimates of flag positions, which the follower drones used to rank sectors according to a utility measure. The utility traded off two elements: the probability of a flag existing in a sector and the spatial distance from other drones. This double consideration guaranteed effective coverage and flag capture, examined on the sector level for simplicity, and we effectively applied this strategy across both simulation and hardware stages.

3.1.1.2 Problem Setup

Mathematically, our problem setup is as follows:

- *Arena:* Our arena is divided into a grid of $m \times n$ sectors, which is defined by the set $S = \{s_{ij} | i \leq m, j \leq n\}$, with s_{ij} corresponding to the sector in the ith row and j^{th} column. Each sector has value p which corresponds to the probability of a flag being in that sector. Hence $p_{ij} \leq 1$ for each sector s_{ij}.

- *Players:* The set of follower drones is defined as $D = \{d_1, \dots, d_k\}$, where k is the total number of drones. For each drone d_a, its position is denoted by x_a, with $x_a \in S$. Multiple drones can occupy the same sector, i.e., for $a \neq b$, x_a can be the same as x_a.
- *Local Neighborhood:* The local neighborhood of a sector $L(s)$ contains all of the sectors within the immediate neighborhood of a sector $s \in S$.
- *Action Set:* Each drone d_i has, at any given iteration, the following action set $A_a = \{a_l | a_l \in L(x_a)\}$
- *Utility:* The utility of a drone d_a in sector s_{ij} is denoted by $u(s_{ij}, d_a)$, which is defined as follows:

$$u(s_{ij}, d_a) = \begin{cases} 0 & s_{ij} \in L(x_{-a}) \\ p_{ij} & otherwise \end{cases}$$

  This means that if a sector lies in the local neighborhood of any drone other than d_a, it has no utility; otherwise, it has the same value as the value of the sector.
- *Algorithm:* We employed the Bilinear Log-Linear (BLL) algorithm for simulation and transitioned to a distributed gradient ascent algorithm for hardware to optimize sector assignments iteratively, adapting the approach to each phase's requirements.

3.1.1.3 Bilinear Log-Linear (BLL)

In the simulation phase, we implemented the BLL algorithm, a variant of Log-Linear learning that restricted decisions to either retain the current action or selecting a random alternative. The process iterated as follows:

- *Step 1:* Randomly select a drone $d_a \in D$
- *Step 2:* Select a random action $a_l \in A_a$
- *Step 3:* Compute the utilities $u(a_l, d_a)$ and $u(x_a, d_a)$
- *Step 4:* Compute softmax between the two values
- *Step 5:* With probability $\frac{e^{u(a_l,d_a)}}{e^{u(a_l,d_a)}+e^{u(x_a,d_a)}}$ select the new action and move to the new location, and with probability $\frac{e^{u(x_a,d_a)}}{e^{u(a_l,d_a)}+e^{u(x_a,d_a)}}$ stay at the same location.

This random selection of the agent and its potential action converges to, at least in theory, the optimal solution after an arbitrary number of iterations.

3.1.1.4 Distributed Gradient Ascent Algorithm

For hardware realization, we adopted a distributed gradient ascent method to guide follower drones in selecting the optimal sectors. This differed from BLL simulation algorithm. While BLL employed probabilistic and random sector selection, the distributed gradient ascent algorithm employed a deterministic exhaustive method better suited for actual distributed operation:

- *Exhaustive Local Search:* Each drone $d_a \in D$ evaluated the utility of all sectors within its local neighborhood $L(x_a)$ where x_a was its current sector. We programmed the drones to perform these calculations in real time, considering up to eight adjacent sectors depending on their position in the grid.
- *Highest Utility Selection:* The drone identified and moved to the sector with the highest utility in $L(x_a)$ abandoning the exploration probabilities of BLL for a more decisive approach. This ensured quicker convergence in the physical environment where random moves could waste flight time.
- *Distributed Planning:* All the drones performed these calculations locally, from the shared probability map and real-time updates from peers' positions. We did not need a central controller, and the system was robust even when one drone momentarily lost communication.

The utility function remained consistent with the simulation:

$$u(s_{ij}, d_a) = \begin{cases} 0 & s_{ij} \in L(x_{-a}) \\ p_{ij} & otherwise \end{cases}$$

This incentivized drones to target high-probability sectors while avoiding overlapping, and we fine-tuned the weighting of p_{ij}on the bases of field test observations.

3.1.2: Simulation Implementation

We ran the experiment on the ROS Noetic and Gazebo, modeling the arena as a grid of sectors. Our experiment involved a simulated leader drone that computed a probability map of flag locations and shared it with follower drones. The drones employed the BLL algorithm to iteratively select the sectors, balancing flag probability and spatial distribution. We ran various experiments to test the algorithm's performance by varying parameters such as grid size and the number of drones for attaining robustness. Implementation validated our theoretical approach and provided us with significant insights that guided the subsequent hardware phase.

3.1.3: Hardware Implementation

Based on the simulation, we implemented the multi-agent flag capture system on hardware. We used one 500mm quadcopter as the lead drone and two 350mm quadcopters as follower drones. We equipped each drone with an ODROID XU4 single-board computer and Ubuntu and ROS Noetic operating system, running control algorithms based on Python. We used Wi-Fi router for communication and UDP protocol to establish communication and form a distributed network for real-time data exchange among the drones. Real Sense Cameras on each drone were employed for depth and color imaging for localizing the drones and detecting the flag. The leader drone localized the arena to generate and broadcast a probability map of flag positions to the follower drones by using simple path and computer vision to detect the flags.

3.2: FINAL DESIGN

The final design locked in our multi-agent system, ready to tackle flag capture outdoors after rounds of testing and tuning.

3.2.1: Drone Configurations

- Leader Drone (500mm Quadcopter):
    - Size/Weight: 500mm frame, 2.5kg loaded.
    - Specs: 30-min flight time, 700KV motors, 12-inch props.
    - Gear: HolyBro Pixhawk 6C, Go Pro Camera, Jetsen Nano, 6S battery, Radio telemetry kit.
- Follower Drones (350mm Quadcopters):
    - Size/Weight: 350mm frame, 1.2kg loaded.
    - Specs: 15-min flight time, 1400KV motors
    - Gear: Matek Mateksys Flight Controller H743-Mini V3, Real Sense, ODROID XU4, 3S battery, Radio telemetry kit.

3.2.2: System Integration

We Assembled the drones, paired each with the router, and used UDP protocol and global broadcast setting to connect each drone to the other on the same access point. And with the same local network we assigned each a drone ID to identify each correctly and to avoid in air collision by incorrect assignment of movement controls. We used SSE lawn and cricket ground where we set up our grid and conducted multiple trials.

3.3: TOOLS/INSTRUMENTS

3.3.1: Simulation Software Packages

- Virtual Machine with Ubuntu 20.04
- ROS Noetic
- Gazebo
- Python

3.3.2: Hardware Instruments

- 350mm quadcopters
- 500mm quadcopter
- Radio Telemetry kits
- Batteries
- Real Sense Cameras
- Go Pro camera
- Wi-Fi Routers
- ODROID XU4

Chapter 4

PROJECT SIMULATION

In this section, we present our simulation environment that validates our algorithmic approaches in real-time conditions. Our simulations closely mirror our hardware setup, ensuring high fidelity results. The following subsection details the simulation configuration.

4.1: SIMULATION COMPONENTS

Our simulation framework integrates Robot Operating System (ROS) with the Gazebo simulator to create a comprehensive testing environment featuring quadcopter drones and goal markers. The drones incorporate a camera plugin paired with a custom-trained YOLO model for marker detection. We implement Software-In-The-Loop (SITL) simulation and MavLink protocols through ROS to control drone movement via Python scripts. These scripts, deployed as ROS Python nodes, also execute our algorithmic implementations. Our entire simulation architecture consists of the following components:

- *Robot Operating Software (ROS):* Our simulation infrastructure relies on ROS Noetic, a robust open-source robotics middleware platform specifically chosen for its compatibility with Ubuntu 20.04 and Python 3.8 integration **[14][15]**. It provides the essential communication framework, resource allocation, and process management capabilities necessary for our quadcopter simulations. To establish bidirectional communication between ROS and our simulated aerial vehicles, we implement MAVROS – a specialized ROS package that functions as an intermediary translation layer, creating a seamless interface between the ROS ecosystem and vehicles operating on the MavLink protocol **[16]**. This critical middleware converts vehicle telemetry data—including precise GPS coordinates, real-time battery levels, current flight modes, altitude readings, and orientation parameters—into structured ROS topics and parameters, which are then accessible through standardized ROS communication mechanisms. This also allows our custom Python scripts to monitor vehicle status, issue control commands, and implement algorithmic logic through a unified programming interface, enabling high-fidelity

simulation while maintaining programmatic control equivalent to our physical hardware implementation.

- *Gazebo (version 11):* Gazebo is a powerful 3D dynamic physics simulator with the ability to accurately and efficiently simulate populations of robots at a very high degree of fidelity, a suite of sensors, and interfaces for both users and programs **[17]**. Within the overarching ROS framework, we use Gazebo to generate a 3D environment suitable for our simulation setup. Within Gazebo, we create a textured, gridded arena with custom textured goal markers, as well as models for our UAVs, which we then manipulate accordingly. We deploy simulated quadcopter UAV models equipped with the *libgazebo_ros_camera.so* plugin for seamless ROS camera integration. The virtual camera system operates at 640×640 pixel resolution with a horizontal field of view of 1.2 radians and clipping ranges of 0.1m to 1000m. **Figure 4** and **Figure 5** show our grass textured simulation arena featuring custom markers distributed across randomly selected sectors.
- *Software-In-The-Loop (SITL):* SITL allows autopilot software such as ArduPilot to run without physical hardware, significantly cutting costs and accelerating development. It enables realistic simulation of various vehicle types—including ArduCopter in our case—and complex scenarios like GPS failure **[18]**. SITL integrates seamlessly with ground control stations such as MAVProxy, providing real-time monitoring and control of flight simulations. Communication between SITL, the autonomous flight control software, and ground stations is handled through MAVLink Router protocols **[19]**, ensuring efficient data exchange and coordination.
- *YOLOv8:* We implement two complementary marker detection approaches: color-based detection and object detection using YOLO. Initially, we developed a color detection system capturing frames from the drone camera and converting them to HSV color space. This process involved defining precise red color ranges, creating color masks, applying morphological transformations, and detecting contours for marker identification. While functional in controlled simulated settings, this method faced significant challenges including sensitivity to lighting variations, limited viewing angle effectiveness, and inconsistent performance across different terrain environments. **Figure 6** shows our initial empty simulation setup with a red cube marker. To overcome these limitations posed by simple color detection, we supplemented our approach with a custom object

detection model instead. We designed a distinctive 5m×5m×2m marker (**Figure 7**) and created an image dataset comprising of 40 screenshots of the marker from various angles within the Gazebo environment (**Figure 8**). After processing this dataset through Roboflow **[20]**, we trained a YOLOv8 Ultralytics model **[21]** locally for 150 epochs, achieving over 87% detection confidence for all test images (**Figure 9**).

- *Algorithm:* Our system integrates Python with ROS and Gazebo through a ROS Python Node to execute multi-drone coordination. The leader drone conducts a systematic row-by-row traversal of the arena, detecting all markers to populate a comprehensive sector matrix that serves as the foundation for subsequent operations. This matrix generates a heat-map visualization representing marker distribution across the environment. Following the leader's complete reconnaissance, follower drones launch from randomized positions and navigate according to our multiagent traversal algorithm until they converge on and detect the markers, demonstrating the system's capacity for effective path planning regardless of initial positioning.



Fig 4 Orthographic view of simulation environment

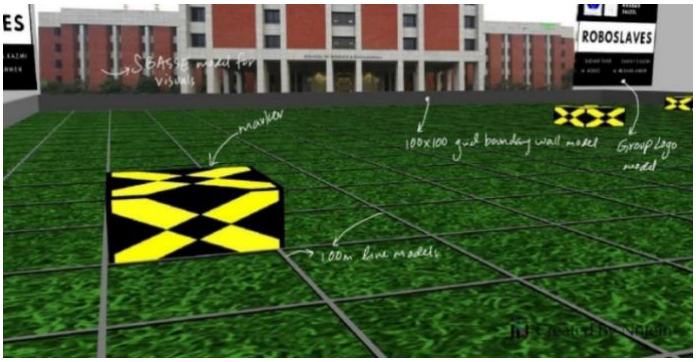
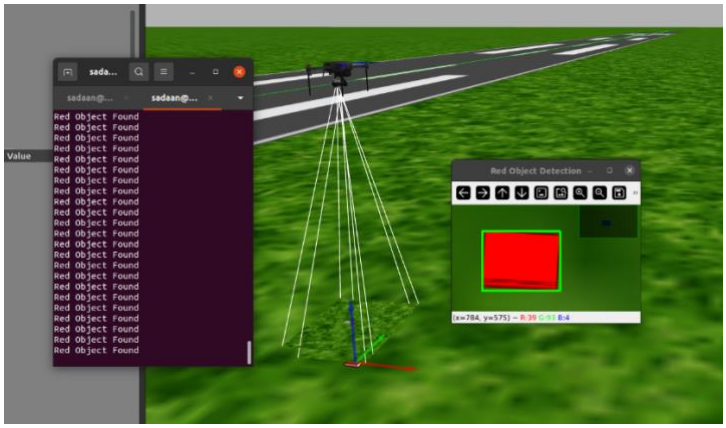


Fig 5 Annotated simulation environment


Red Object Found
Red Object Found
Red Object Found

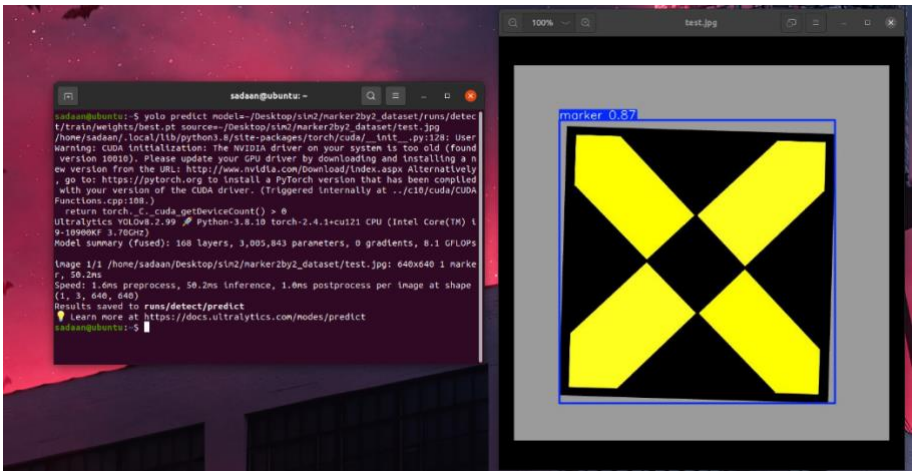


Fig 9 Testing custom trained YOLOv8 model. 87% accuracy is achieved with only 40 train images on the test image shown.

Figure 10 below shows a systematic flow diagram of our simulation architecture. It shows how MAVROS bridges drone operations via MAVLink Protocol with SITL while our Python algorithm executes in Gazebo, all integrated through a ROS Python node enabling efficient data exchange and control.

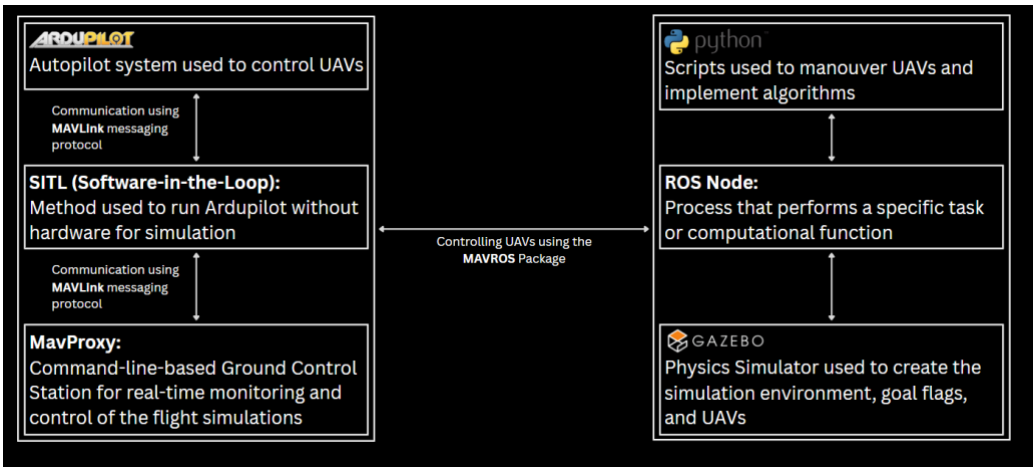


Fig 10 Systematic Flow Diagram of our Simulation Architecture

For our experiment, we constructed a 20m × 20m sectored arena in Gazebo with uniform 5m × 5m sectors. The simulation employs one leader drone and three follower drones, with three markers strategically placed for individual detection tasks. The system runs on Ubuntu 20.04

with ROS Noetic integration, utilizing Python 3.8 scripts for compatibility. All simulations are executed on an RTX 3050 Twin Fan MSI GPU to ensure computational efficiency.

4.2: SIMULATION RESULTS

Our simulation experiments demonstrated successful marker detection and traversal using the BLLL algorithm. The leader drone effectively conducted complete arena traversal, precisely identifying all markers and generating accurate positional data for each marker. **Figure 11** below illustrates the resulting heat map with adequately precise marker location estimation by the leader drone. Remarkably, the BLLL traversal algorithm enabled the follower drones to converge on optimal locations within just 153 iterations. **Figures 12-14** document the successful marker detection events by the three follower drones upon reaching their designated targeted marker proximities.

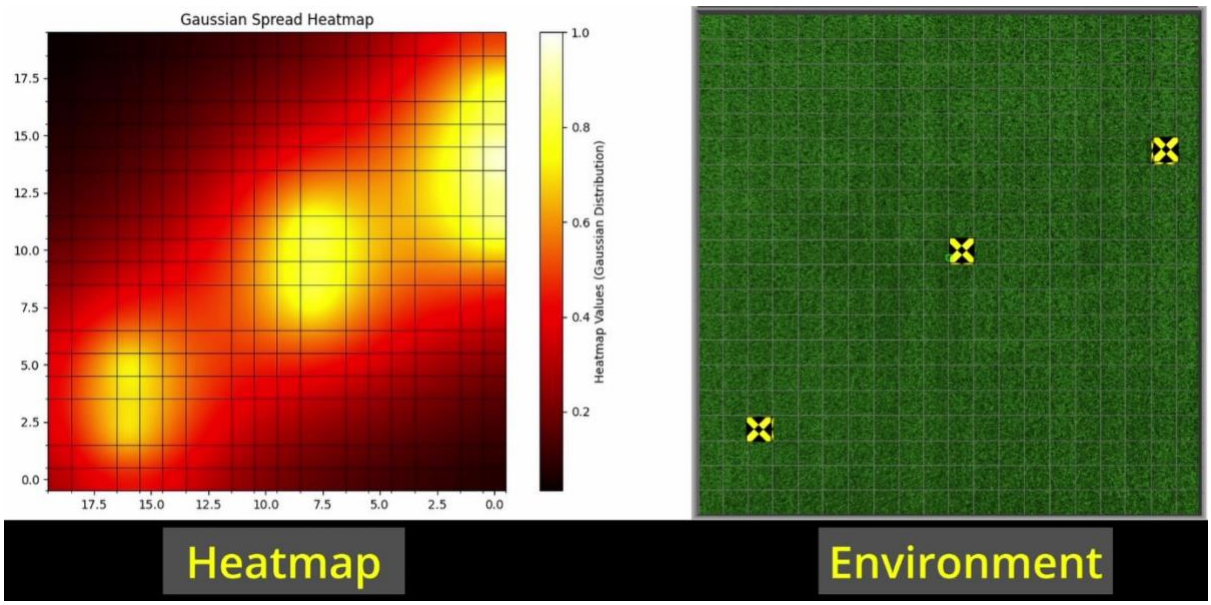


Fig 11 Location of markers on gridded arena and generated heatmap by leader drone

The simulated experimental results confirm BLLL's efficacy as an optimization algorithm for drone convergence on target markers, validating its suitability for real-world implementation. Additionally, YOLO's robust performance in marker detection establishes it as a reliable foundation for our computer vision system in physical deployment scenarios.

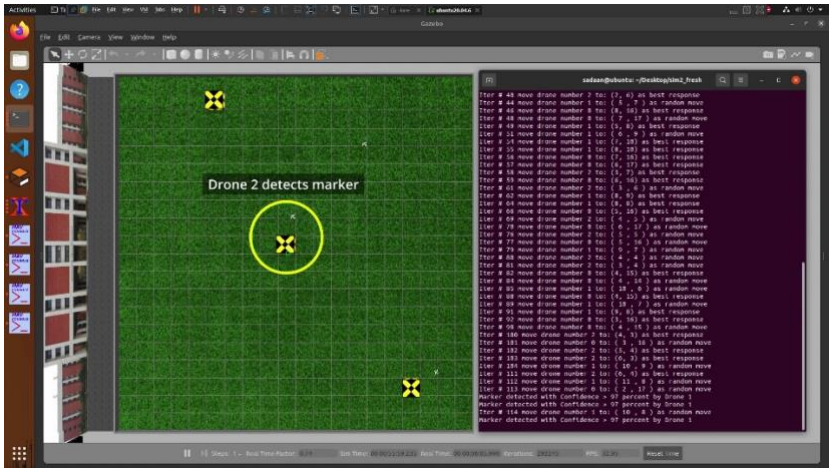


Fig 12 Drone 2 detects its targeted marker

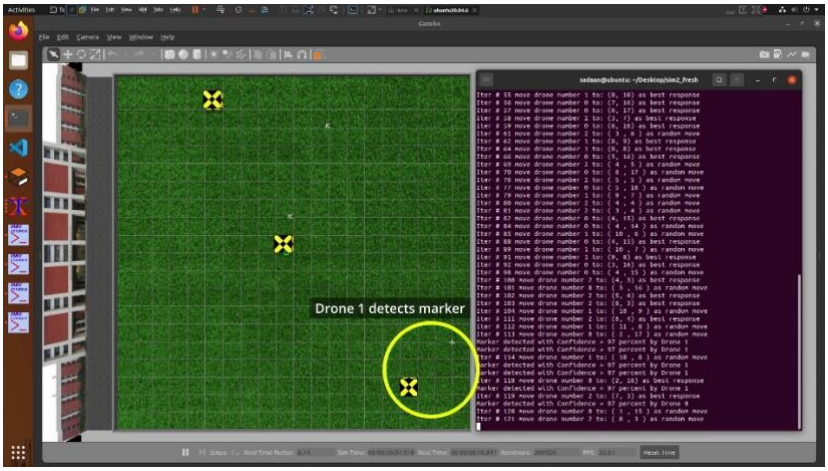


Fig 13 Drone 1 detects its targeted marker

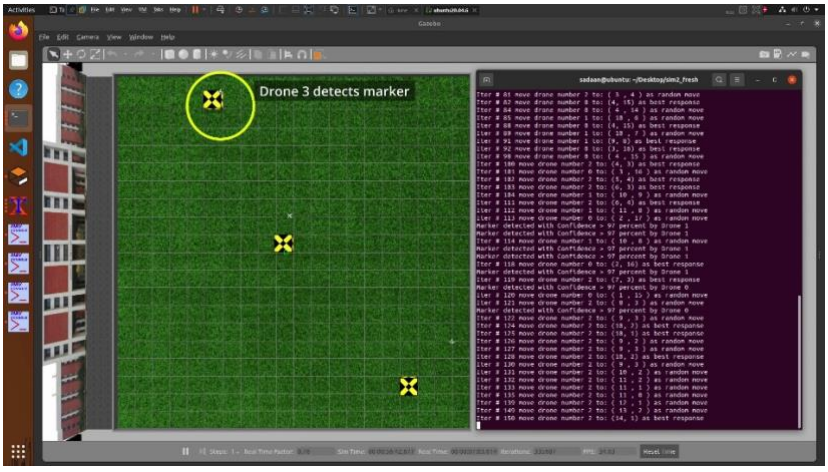


Fig 14 Drone 3 detects its targeted marker

Chapter 5

PROJECT IMPLEMENTATION

5.1: HARDWARE SETUP

With the completion of our simulation setup, we begin with the hardware implementation of our multiagent UAV system. **Figure 15** below presents a systematic flow diagram illustrating the comprehensive integration of our custom-built drones - consisting of one leader and three follower units - with the project's core objectives. We divide our complete hardware implementation into three phases. Phase 1 can be defined as leader traversal where the leader drone is tasked autonomously mapping of a 20m x 20m grass filled grid outlaid with randomly placed markers. The number of markers corresponds directly to the number of followers drones available in Phase 2, that is termed as follower traversal. In Phase 2, the follower drones' takeoff from random positions from within the gridded arena and our game theoretic algorithm enables them to reach near the vicinity of the markers, which is precisely when Phase 3 triggers. Phase 3 is essentially termed as color landing as once near the vicinity of a marker, each of the follower drones deploy an autonomous landing algorithm based off entirely color masking and Proportional-Integral-Derivative (PID) control to land on top of the marker.

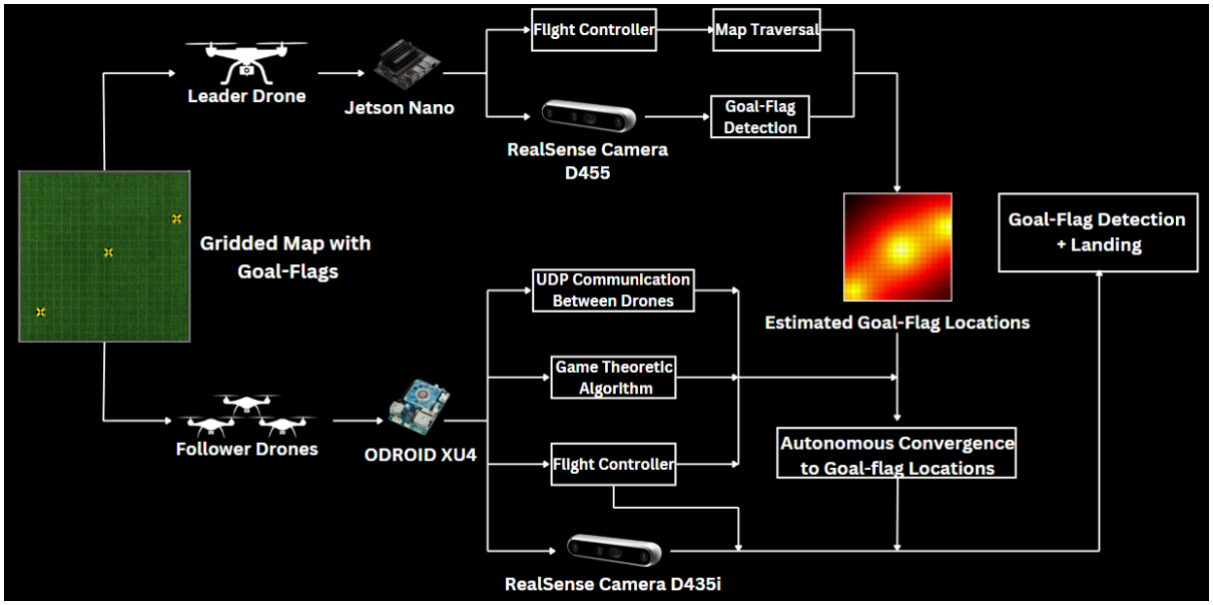


Fig 15 Systematic flow diagram for the hardware implementation

We designed and assembled a fleet of five UAVs consisting of two distinct frame configurations. **Table 2** below detail the specific electronic components integrated into each drone type, highlighting the architectural differences between the two designs.

Table 2 Electronic components for each drone type

Leader Drone (500mm frame type)	***Follower Drone (350mm frame type)***
500mm carbon fiber frame	350mm carbon fiber frame
4-in-1 55A ESC	4-in-1 40A ESC
1400KV brushless motors	700KV brushless motors
12×4.5 inch plastic propellers	8-inch plastic propellers
14.8V 5200mAh 4S 45C LiPo battery	11.1V 2200mAh 3S LiPo battery
3DR 433MHz radio telemetry kit	3DR 433MHz radio telemetry kit
Pixhawk 6C flight controller module	Matek H743-Mini flight controller with BDSHOT300 support
Jetson Nano Orin	Odroid XU4 - single board Computer
Intel RealSense D455 camera module	Intel RealSense D435i camera module
Matek M10Q-5883 GPS, Digital Airspeed with Compass Module	Matek M10Q-5883 GPS, Digital Airspeed with Compass Module

We assembled one leader drone (500mm) and four follower drones (350mm), mounting all components onto their respective carbon fiber frames. Each drone's electronics, including ESCs, motors, flight controllers, power modules, and telemetry kits, were soldered and securely integrated. We configured all five drones using ArduPilot, performing full IMU calibration, compass setup, radio calibration, and ESC/motor orientation checks. Each system was thoroughly tested for sensor errors and calibrated for stable flight. The leader drone used a Pixhawk 6C with a Jetson Nano Orin, while follower drones used Matek H743-Mini controllers with Odroid XU4s.

Figure 16 below shows the fully assembled, flight-ready leader drone, while **Figure 17** displays the completed follower drones. For the follower drones, we designed laser-cut ABS plastic housing for the IMU/compass module. This significantly improved stability by allowing the Odroid XU4 onboard computer to be mounted directly above the flight controller, rather than hanging off the side as in our initial hardware setup. The streamlined configuration reduced drag, and the custom housing securely enclosed the compass module for added protection. **Figure 18** highlights the laser-cut cover and provides a close-up view of one of the deployed follower drones.



Fig 16 Leader drone (500mm frame type)



Fig 17 Follower drones (350mm frame type)

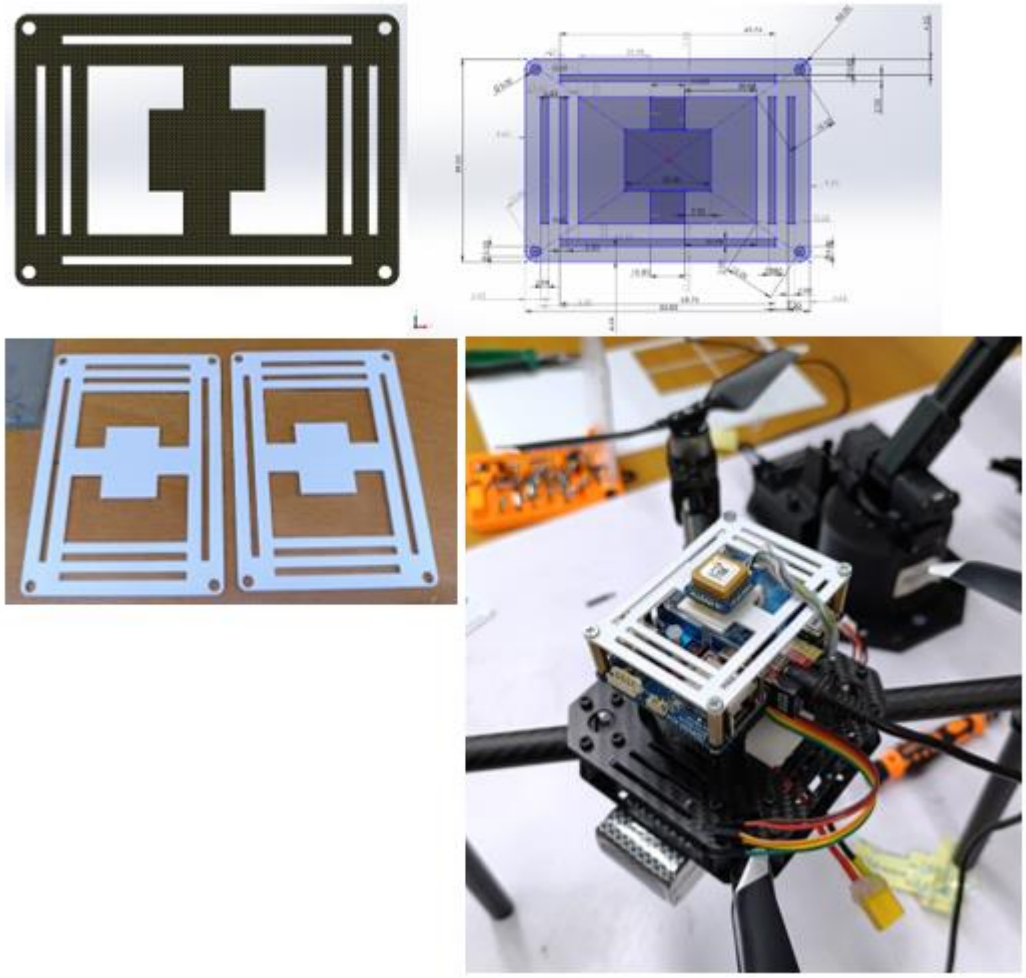


Fig 18 ABS sheet cover

We constructed two 80cm × 80cm markers using plywood sheets supported by shallow wooden legs. Custom marker textures were printed and affixed to the top surfaces. For the experimental procedure, only two follower drones were deployed alongside the two markers constructed. **Figure 19** below shows the marker texture used and one of the two markers.



Fig 19 The constructed marker with our custom texture

Our experimental testing was conducted at the LUMS cricket ground. **Figure 20** shows an annotated top-down view (red square) from Google Earth of the 20m × 20m gridded arena we created. Plastic sport cones were used to mark the grid boundaries, as seen in **Figure 21**. Moreover, to establish communication between the drones, we used a TP-Link AC1200 Gigabit Wi-Fi router.



Fig 20 The red square marks the boundaries of our testing area



Fig 21 Experimental setup

5.2: EXPERIMENTAL PROCEDURE

Our hardware deployment was conducted across the Spring Semester. Incrementally, we tested the functionality of our drones, the ability to manipulate them through software, our ability to receive and utilize sensory data, robustness to environmental factors, and full algorithmic deployment.

These experiments took place either in the ground outside the Syed Babar Ali School of Science and Engineering (SBASSE) or in the LUMS Cricket Ground. As the semester progressed, these experiments were increasingly shifted to the Cricket Ground, accounting for more space, safety of the public, and better satellite signal reception for GPS coordinates.

For the final experiment, we created a 20m x 20m square arena in the Cricket Ground. We defined the boundary of this area using space markers and placed our goal-flags within this area at locations corresponding to specific sectors within the grid. Fig 15 shows the entire setup of the arena.

Broadly, our deployment can be categorized into 3 phases:

- Leader Traversal
- Follower Traversal
- Color Landing

5.2.1: LEADER TRAVERSAL

The leader drone, following a Python script, takes off from one corner of the gridded arena. It will then proceed to traverse the entire arena in a boustrophedon path, as shown in Fig 22. It moves along the entire length of the grid, takes a turn at the end, and moves slightly in the perpendicular direction, before turning and traversing back along the same length of the grid.

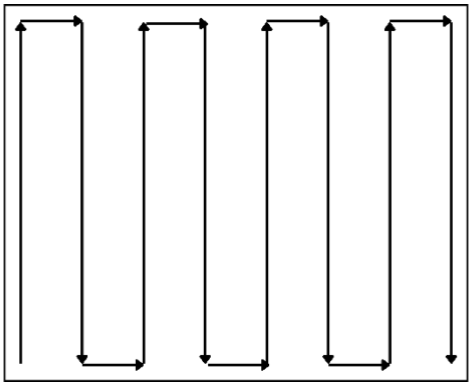


Fig 22 Diagram Illustrating Boustrophedon Path

At the same time, the RealSense camera attached# to the drone is continuously operational and provides visual feedback during traversal. This, in tandem with the YOLO algorithm running in the background, is responsible for detecting the goal-flags within the arena. As the drone moves around the arena, it detects the flags and notes their location as it keeps on moving.

Once the traversal is complete, the drone autonomously lands in the last corner of the grid. It then uses the data it has collected about the markers to generate the heat map, which is to be used by the follower drones in the next phase.

5.2.2: FOLLOWER TRAVERSAL

In contrast to the simulation, the hardware experiment utilizes two follower drones and two markers for the follower traversal. The follower traversal is comprised of the following crucial components:

- *Communication between Drones:* As illustrated in Fig 23, all the drones are connected to the same Wi-Fi router placed near the arena. As a result, the drones can be accessed with laptops via Remote Desktop in the case of emergencies.

  The drones themselves communicate commands and information about their positions amongst each other using a UDP communication established over the Wi-Fi connection. In this hardware setup, all the algorithmic computation is being done on one of the two drones (labelled as “Drone 1” in Fig 23), which then communicates the instruction to the other drone via this connection.

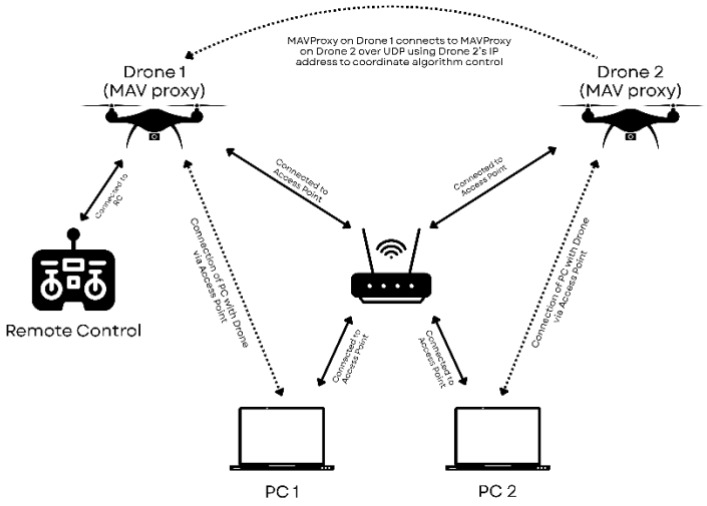


Fig 23 Diagram Illustrating Communication between Drones

- *Game Theoretic Algorithm:* The algorithm used for the hardware deployment is the Distributed Gradient Ascent (DGA), whose details are laid out in Chapter 3. Via GDA, on each iteration, the drone chooses the best possible action within its local neighborhood. It is a much more straightforward approach than the explorative BLLL algorithm, and has been adopted to account for the limited flight time of the drones.

  As mentioned above, the entire algorithmic computation is being done on one drone only. This has also been done to ensure power efficiency in light of the battery constraints of the drones. This is still a multi-agent algorithm; Drone 1 does not act as a central planner but instead, does Drone 2's calculations on its behalf. It then sends the command for the move that Drone 2 needs to play at every iteration, in lieu of DGA, via the communication architecture laid out above.

5.2.3: COLOR LANDING

Once the algorithm ensures that the drones reach the vicinity of the marker, the drones then proceed to land on their respective markers using visual feedback from the RealSense cameras that are attached to them. Once the marker is in the field of vision of the camera, the drone automatically starts to track the marker. It tries to center itself at the small red square at the center of the marker. It employs a proportional controller to manipulate the parallel and perpendicular

motions of the drone for the centering process. Once the centering is complete, it then proceeds to land on the marker.

5.3: IMPLEMENTATION RESULTS

This section outlines the outcomes of the real-world implementation of the decentralized drone system, covering each of the three key phases: Leader Drone survey, Follower Drone coordination, and Color-based Landing. All implementations were conducted in a 20x20 meter arena, with the drones operating at carefully selected heights to match practical and logistical requirements. Videos illustrating each phase are attached accordingly.

5.3.1: Leader Drone Exploration

The implementation began with the Leader Drone stage. The drone was sent for an aerial scan of the arena with a fixed altitude of 7 meters. Two target markers (flags) were in the arena, and the task for the Leader Drone was to detect them in a Boustrophedon Path, a high-speed and methodical sweeping approach designed to maximize coverage.

During this traversal, the drone utilized a YOLO-based object detection model, which was trained on RoboFlow to locate the colored markers. The model was tested on a held-out test set, and performance metrics are shown in Figure 9. And the deployed model showing its working is shown in Figure 24

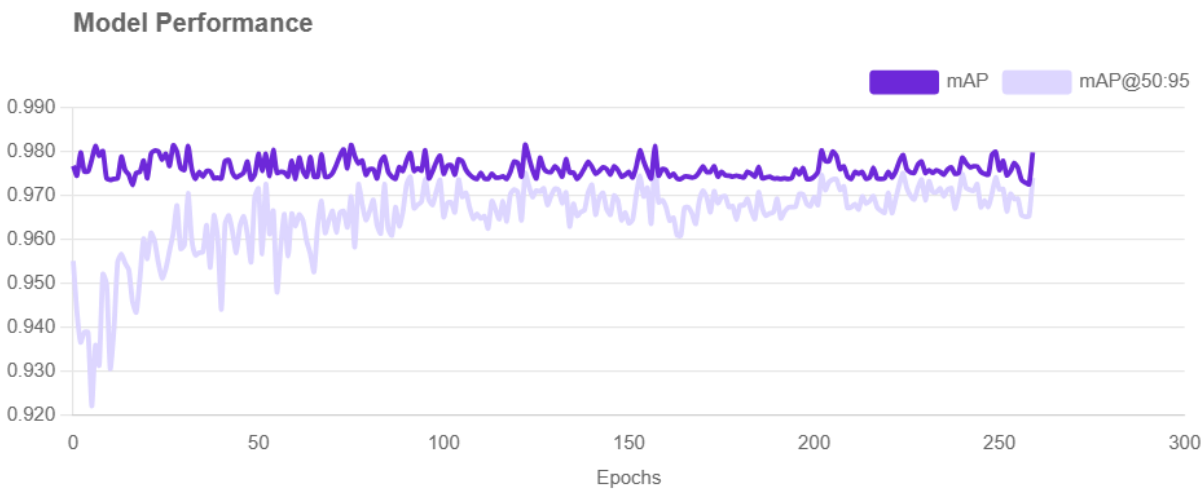


Fig 24 Model Performance

Once the markers were detected as shown in figure 25, the drones' GPS coordinates were noted, and a probability distribution heatmap was created to represent possible target regions. This output is shown in Figure 26, indicating the completion of Leader Drone's mission. The complete utilization of this phase is illustrated in the video that can be by clicking here.

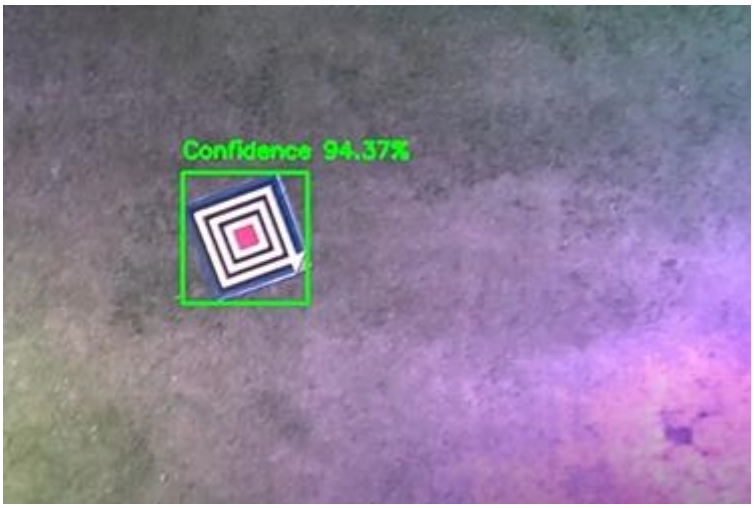


Fig 25 Marker Detection

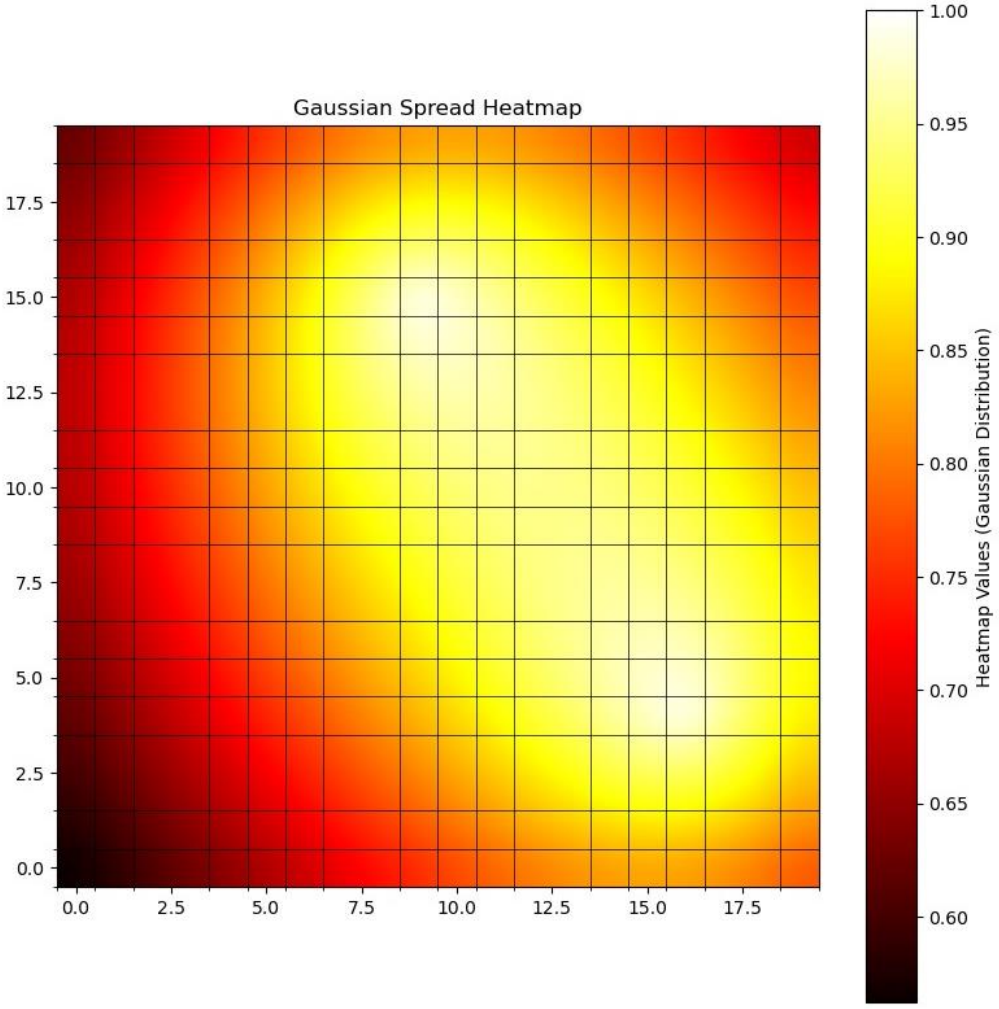


Fig 26 Heatmap Generated by Leader Drone

5.3.2: Leader Drone Exploration

The second phase involved the sending of two Follower Drones with limited capabilities due to hardware constraints and availability of telemetry modules. These drones launched together and flew at the same altitude of seven meters.

The drones executed the Distributed Gradient Ascent algorithm, enabling them to independently calculate utility values from the Leader's heatmap and move towards the optimal unexplored positions. The utilities were calculated in a decentralized environment, but for hardware implementation, a central planner (Odroid board) was used for both drones, owing to physical connectivity limitations.

In physical deployment, the drones moved towards the flags depending on utility gradients. Although absolute alignment with markers was not feasible at times because of GPS inaccuracies and communication jitter, both the drones successfully reached 3–5-meter vicinity of the markers. Their relative proximity to the targets confirms the robustness of the coordination algorithm in the presence of real-world noise and drift.

The video of this implementation can be viewed by clicking here, which demonstrates the drones executing decentralized traversal and target alignment.

5.3.3: Color Detection and Landing

For the last stage, precision landing and visual marker detection, a drone was deployed employing Camera-based positioning. The drone kept a height of 5 meters while in its scan and detection phase.

The detection was carried out using simple color masking, isolating the red marker in the drone's camera feed. A PID controller was used to guide the drone's center over the detected marker and maintain alignment. Once aligned, the drone gradually reduced its altitude while continuously adjusting its position until it touched down on the flag.

The complete operation, starting from detection and landing, took around 10 seconds. The execution was a success, with the drone landing in high visual precision on the marker. The video of this execution can be seen by clicking here.

5.4: RESULT ANALYSIS AND OUTLOOK

The goal of this project was to implement a decentralized, multi-drone system capable of efficient area exploration, autonomous coordination, and precise landing using visual markers — all while remaining resilient to real-world constraints such as limited GPS accuracy and communication jitter.

Each phase of the implementation is analyzed below with respect to these objectives:

5.4.1: Leader Drone: Exploration and Marker Detection

Efficiently scan the environment and identify marker locations for downstream coordination. The table below shows the results.

Table 3 Leader Drone Performance

Metric	**Result**	**Comments**
Coverage Time	3 minutes	Slow to show implementation but within practical bounds for real-time operations if hard coded delays removed
Detection Accuracy (YOLO)	94.4% confidence (Figure 10)	High accuracy confirmed test flight
Altitude	7 meters	Balanced height for coverage + detection

The utilization of the Boustrophedon Path allowed for a rapid and non-redundant sweep of the arena by the drone. The YOLO-trained model using RoboFlow attained high accuracy for detection in identifying red markers within the flight window. The heatmap output, derived using GPS-tagged detections, successfully guided later drone decisions, confirming this stage.

5.4.2: Follower Drones: Distributed Coordination

Autonomously distribute drone agents to unexplored areas and converge near target locations using local utility computation.

Table 4 Follower Drones Performance

Metric	**Result**	**Comments**
No of Drones	**2**	**S**imulations used 3–5 drones for testing
Distance to nearest marker upon landing	**3.5 m and 5 m (average)**	Within acceptable error margin
Algorithm Used	**Distributed Gradient Ascent**	Efficient, despite central planner used
Altitude	**7 meters**	Standardized with Leader Drone

While the implementation wasn't fully decentralized because of telemetry constraints, Gradient Ascent reasoning was properly used on every step. The drones responded to utility changes and moved in accordance, leading to proximity towards the target flags.

5.4.3: Visual Detection and Landing

Accurately detect colored landing marker and safely land

Table 5 Color Landing Performance

Metric	**Result**	**Comments**
Detection Method	**HSV color masking (red)**	Simple and computationally lightweight
Time to complete landing	**~10 seconds**	Fast and consistent
Final landing error	**< 0.5 m (visually observed)**	Acceptable error in manual inspections
Altitude before landing	**5 meters**	Lower height for better detection

The PID-controlled landing mechanism facilitated the drone's direct landing on the red flag with negligible horizontal drift. The controller actively compensated for orientation and descent velocity in real time, depending on visual centering error.

5.4.4: Outlook and Future Work

The current system successfully demonstrates the fundamental framework of a decentralized drone swarm. With the limited hardware resources and environmental conditions, the drones accomplished the goals with satisfactory quantitative performance and consistent results.

- **True Decentralization**: Replace the central planner with an onboard Odroid or companion computers for each drone.
- **Robust Detection Models**: Upgrade to deep segmentation models to handle variable lighting and marker degradation
- **Improved GPS integration**: Introduce GPS for centimeter-level localization accuracy. Or remove GPS as a whole for the follower drone phase.
- **Scalability**: Increase the number of follower drones to 5+ and test in more complex environments.
- **Landing Feedback**: Integrate optical flow or visual odometry to assess landing precision quantitatively.

Chapter 6

COST ANALYSIS

Table 6 Imported Drone Parts List

Sr #	*Description*	*Rate (PKR)*	*Quantity*	*Total Cost (PKR)*
1	**Flyroun LX350 Frame 350mm**	**30995**	**4**	**123982**
2	**DJI Mavic Pro Motors (1400 KV)**	**16916**	**4**	**67664**
3	**ESC 4 in 1 40 A (JHEMCU EM40A)**	**15546**	**4**	**62183**
4	**Propellers 8 Inch, 8331 Size (8 Propellers/package)**	**9797**	**4**	**39187**
5	**Hereflow Optical Sensor**	**41987**	**4**	**167949**
6	**TF Miniplus 1D LiDAR Range Finder Sensor**	**23061**	**4**	**92244**
7	**Skydroid T10 Remote Controller**	**45702**	**1**	**45702**
8	**Skydroid T10 Receiver**	**28572**	**1**	**28572**
9	**Flysky FS-i6X 2.4GHz 10CH AFHDS 2A RC Transmitter with FS-IA6B FS-X6B FS-iA10B Receiver**	**29210**	**1**	**29210**
10	**MATEK Mateksys FLIGHT CONTROLLER H743-MINI V3**	**38986**	**4**	**155946**
11	**Matek M10Q-5883 GPS, Digital AirSpeed With Compass Module**	**19067**	**4**	**76270**
12	**CPT DC Converter 12V/24V to 5V Buck Converter 5A 25W**	**8468**	**4**	**33873**

13	**HolyBro Pixhawk 6C PM02 M10**	**113574**	**4**	**454296**
Total				**1,377,078**

Table 7 Local Drone Parts List

Sr #	**Description**	**Rate (PKR)**	**Quantity**	**Total Cost (PKR)**
1	**X4 Carbon Fiber Frame 500mm**	**16240**	**2**	**48720**
2	**X4 carbon fiber landing frame**	**4984**	**1**	**4984**
3	**ESC 4 in 1 55 A JHEMCU**	**11760**	**4**	**47040**
4	**Carbon fiber magnetic propeller balancer**	**728**	**1**	**728**
5	**Velcro battery step 300mm 1 feet**	**134**	**10**	**1340**
6	**XT60 battery male plug with silicon 14 AWG Wire**	**504**	**5**	**2520**
7	**Silicon wire 12AWG black**	**448**	**6**	**2688**
8	**3s 2200mah 30c CNHL Black series lipo battery**	**6496**	**2**	**12992**
9	**GARTT ML3508 700KV Brushless motor 2kg thrust**	**728**	**4**	**2912**
10	**Propeller 12x4.5 prop12 inch [plastic]**	**1344**	**2**	**2688**
11	**Carbon fiber propeller 12x3.8**	**2520**	**4**	**10080**
12	**Buck converter 5V 3A**	**896**	**2**	**1792**
13	**Gens Ace 2200mAh 3S 11.1V Battery (XT60)**	**25984**	**1**	**25984**
14	**Type C OTG**	**9632**	**1**	**9632**

15	**ZOP Power 14.8V 5200mAh 4S 45C Lipo Battery XT 60**	**9504**	**4**	**38016**
16	**8mp Raspberry Pi Camera Module V2**	**392**	**5**	**1960**
Total				**300,623**
Tax				**34,880**
Grand Total				**335,503**

Table 8 Other Additional Components

Sr #	*Description*	*Rate (PKR)*	*Quantity*	*Total Cost (PKR)*
1	**Rtx 3050 twin fan**	**90000**	**1**	**90000**
2	**Buck Converters**	**1875**	**4**	**7500**
3	**Miscelliniuos**	**-**	**-**	**10000**
Total				**107,500**

6.1: Cost Breakdown of the Proposed Drone System

The total cost of the system is derived from imported and locally sourced components (Tables 5,6 and 7):

- Total Hardware Cost: PKR 1,377,078 (imported) + PKR 335,503 (local) + PKR 107,500 (additional) = PKR 1,820,081 (~$6,450 USD).

6.2: Comparison with Traditional Helicopter-Based Synthetic Aperture Radar (SAR)

Helicopter emergency medical services (HEMS) and SAR operations are significantly more expensive due to:

- **High Fixed Costs**:
    - Helicopters incur ~93% fixed costs (e.g., 24/7 staffing, maintenance) **[12]**
- **Operational Limitations**:
    - Helicopters cost ~$1,200–$3,000/hour to operate on average. **[13]**

6.3: Cost-Effectiveness of Drone Swarms

The proposed system offers advantages over both helicopters and single-drone solutions:

1. **Scalability**:
    - Adding drones (PKR 300K–500K/unit) is cheaper than deploying additional helicopters (PKR 3M–10M/unit).
2. **Efficiency**:
    - Simulation Results: The BLL algorithm achieved target convergence highlighting minimizing search time [Chapter 4].
    - Field Tests: Drones reduced marker localization time to ~10 seconds [Chapter 5].
3. **Environmental Impact**:
    - Drones have a smaller carbon footprint than helicopters.

6.4: Conclusion

The distributed drone system is 96% cheaper per mission than helicopters and offers superior scalability for SAR. While helicopters remain vital for critical care, drones excel in rapid, low-risk surveillance. Future cost reductions in hardware will further enhance viability.

Chapter 7

CONCLUSION AND FUTURE RECOMMENDATIONS

Our project successfully demonstrated the core objectives of developing an autonomous multi-agent drone system for surveillance and rescue through simulation. We achieved the end-to-end integration of key modules: aerial survey using a master drone, object detection via computer vision, heatmap generation for target probability, gradient-based path planning for swarm deployment, as well as precise color landing of the drones onto the target locations utilizing the camera modules effectively. The use of ROS, Gazebo, Ardupilot, and MAVProxy enabled realistic simulation and real-time validation of our algorithms in a controlled environment. The hierarchical structure—combining a global planner (master drone) and local agents (smaller drones)—proved effective in achieving scalable area coverage and autonomous decision-making, a critical requirement for real-world search-and-rescue applications.

Qualitatively, the project met most of its technical objectives, including distributed path planning, vision-based flag detection, and collision-free multi-agent navigation. The drones exhibited cooperative behavior and demonstrated the ability to converge on high-probability regions effectively. However, some limitations remained due to the constraints of simulation fidelity and hardware integration. Specifically, while the simulation environment allowed for extensive testing, the transfer to real-world hardware was not fully realized within the current project timeline due to resource and time limitations. Real-time computer vision under uncontrolled lighting and dynamic environments remains a challenge to be addressed during hardware deployment.

For future work, we recommend the full deployment of the system on physical drone hardware to test its robustness in dynamic, real-world environments. Enhancing the object detection pipeline with real-time deep learning models optimized for edge devices can improve accuracy and speed. Additionally, integrating inter-drone communication using onboard radios can enable more decentralized and fault-tolerant coordination. Exploring adaptive path planning algorithms and incorporating SLAM-based localization can further increase the system's autonomy in GPS-denied environments. Since battery and payload constraints resulted in limited flight times, we suggest exploring the possibility of a portable solar power system for the drones as a future enhancement. Integrating lightweight, high-efficiency solar panels on the drone's surface could

help extend flight duration by supplementing battery power during daylight operations. This hybrid energy approach would be particularly beneficial for long-duration surveillance missions or operations in remote areas where frequent recharging is impractical. Finally, expanding the mission scope to include dynamic obstacle avoidance and real-time victim identification would bring the system closer to real-life deployment in critical scenarios.

References

[1] A. Albanese, V. Sciancalepore, and X. Costa-Pérez, 'SARDO: An Automated Search-and-Rescue Drone-Based Solution for Victims Localization', *IEEE Transactions on Mobile Computing*, vol. 21, no. 9, pp. 3312-3325, 2022.

[2] H. J. Jaleel and R. M. Murray, 'LP-based Multi-vehicle Path Planning with Adversaries', in *2010 American Control Conference*, 2010, pp. 2641-2646.

[3] H. J. Jaleel, M. A. Saquib, and R. M. Murray, 'Distributed Real Time Control of Multiple UAVs in Adversarial Environment: Algorithm and Flight Testing Results', in *2018 IEEE International Conference on Robotics and Automation (ICRA)*, 2018, pp. 3173-3178.

[4] S. Mayer, L. Lischke, and P. W. Woźniak, 'Drones for Search and Rescue', in *1st International Workshop on Cyber-Physical Systems for Smart Cities and Communities (CPS-SCC 2017)*, 2017, pp. 24-27.

[5] P. Bianchi and J. Jakubowicz, 'Convergence of a Multi-Agent Projected Stochastic Gradient Algorithm for Non-Convex Optimization', in *2013 IEEE Global Conference on Signal and Information Processing (GlobalSIP)*, 2013, pp. 100-103.

[6] J. R. Marden and J. S. Shamma, 'Revisiting log-linear learning: Asynchrony, completeness and payoff-based implementation', *Games and Economic Behavior*, vol. 75, pp. 788-808, 2012.

[7] B. Mishra, D. Garg, P. Narang, and V. Mishra, 'Drone-surveillance for search and rescue in natural disaster', *Computer Communications*, vol. 156, pp. 1-10, 2020.

[8] C. van Tilburg, 'First Report of Using Portable Unmanned Aircraft Systems (Drones) for Search and Rescue', *National Park Service Report*, 2017.

[9] GitHub.com, 'SITL + Ardupilot + Gazebo + ROS Camera plugin Tutorial', 2019. [Online]. Available: https://github.com/r0ch1n/ardupilot_gazebo_roscam . [Accessed: 11- Oct- 2024].

[10] GitHub.com, 'Multi-UAV Gazebo Simulation', 2019. [Online]. Available: https://github.com/monemati/multiuav-gazebo-simulation?tab=readme-ov-file. [Accessed: 11- Oct- 2024].

[11] GitHub.com, 'Gazebo models and worlds collection', 2019. [Online]. Available: https://github.com/leonhartyao/gazebo_models_worlds_collection. [Accessed: 11-Oct- 2024].

[11] GitHub.com, 'Gazebo models and worlds collection', 2019. [Online]. Available: https://github.com/leonhartyao/gazebo_models_worlds_collection. [Accessed: 11-Oct- 2024].

[12] A, A. et al., 'The estimated cost-effectiveness of physician-staffed helicopter emergency medical services compared to ground-based emergency medical services in Finland'. 2024. Available at: https://pubmed.ncbi.nlm.nih.gov/38821704/ [Accessed: 21 May 2025].

[13] Revector-Wbn. 'Reducing costs of Search & Rescue. Revector Providing IMSI and WiFi Intelligence. ' 2024 Available at: https://www.revector.com/reducing-costs-of-search-rescue/#:~:text=The%20operational%20cost%20of%20flying,and%20crew%20in%20these%20operations. [Accessed: 21 May 2025].

[14] ROS.org, 'ROS Noetic Ninjemys', 2024. [Online]. Available: http://wiki.ros.org/noetic. [Accessed: 21-May-2025].

[15] Python.org, 'Python 3.8. Documentation', 2024. [Online]. Available: https://docs.python.org/3.8/ . [Accessed: 21-May-2025].

[16] GitHub.com, 'MAVROS: MAVLink extendable communication node for ROS', 2024. [Online]. Available: https://github.com/mavlink/mavros . [Accessed: 21-May-2025].

[17] GazeboSim.org, 'Gazebo - Robot simulation made easy', 2024. [Online]. Available: https://gazebosim.org/ . [Accessed: 21-May-2025].

[18] ArduPilot.org, 'SITL Simulator (Software in the Loop)', 2024. [Online]. Available: https://ardupilot.org/dev/docs/sitl-simulator-software-in-the-loop.html . [Accessed: 21-May-2025].

[19] GitHub.com, 'MAVLink-router', 2024. [Online]. Available: https://github.com/intel/mavlink-router . [Accessed: 21-May-2025].

[20] Roboflow.com, 'Roboflow: Give your software the eyes of a human', 2024. [Online]. Available: https://roboflow.com/ . [Accessed: 21-May-2025].

[21] GitHub.com, 'Ultralytics: Cutting-edge AI for object detection', 2024. [Online]. Available: https://github.com/ultralytics/ultralytics . [Accessed: 21-May-2025].